UNIVERSIDAD TORCUATO DI TELLA

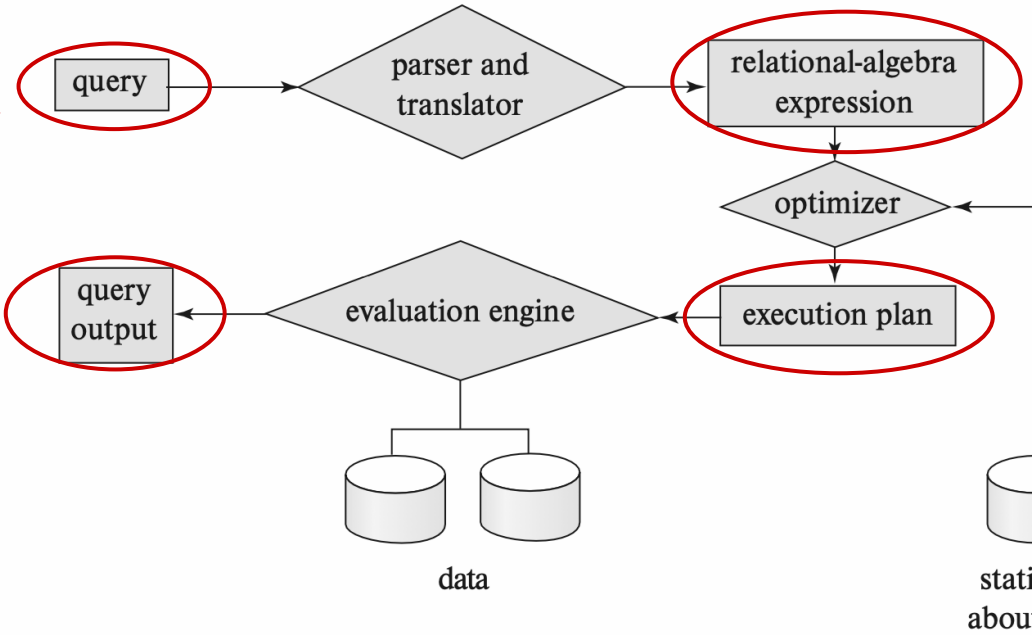
TD7: Ingeniería de Datos

Procesamiento de consultas

Procesamiento de consultas

Procesamiento de consultas

La sentencia SQL ingresada



Una (posible) expresión de álgebra relacional para ese SQL

Un plan de ejecución de la expresión elegida, con estrategias concretas para cada paso

El resultado de la ejecución

Procesamiento de consultas

- El punto de entrada es una **sentencia SQL**. Como lenguaje declarativo expresa el qué pero no el cómo
- Se traduce a una de las posibles expresiones de **álgebra relacional**, lenguaje procedural que indica los pasos a seguir.
- El motor tiene que evaluar entonces distintas expresiones de álgebra relacional equivalentes y distintas estrategias para cada paso, y elegir la que tenga más chances de ser la óptima.

Plan de ejecución

Información de catálogo

- Las diferentes estrategias para ejecutar una consulta van a tener distinto costo, dependiendo de las situación concreta de los datos.
- Por esto, el motor mantiene datos estadísticos sobre las relaciones:
  - n_r: cantidad de tuplas en la relación r.
  - b_r: cantidad de bloques con tuplas de la relación r.
  - l_r: tamaño de una tupla de la relación r (en bytes).
  - f_r: cantidad de tuplas de r que entran en un bloque (factor de bloque).
  - V(A, r): cantidad de valores distintos que aparecen en el atributo A de la relación r (equivalente al tamaño de la relación $\pi_A(r)$).
- Y también de los índices de esas relaciones:
  - $Height_{I(A,r)}$: altura del árbol B+ del índice I sobre el atributo (o atributos) A de r
  - $Length_{I(A,r)}$: tamaño en bloques de las hojas del índice

Plan de ejecución

- Una vez traducido el SQL a expresión algebraica, el motor genera expresiones equivalentes a partir de **reglas de transformación**.
- Se evalúan los costos de las distintas estrategias para cada parte de cada expresión utilizando las estadísticas y una **estimación de costo** específica de cada estrategia.
- Las estimaciones se basan en las dos operaciones más costosas: **seek de bloque** y **transferencia de bloque** y sus respectivas métricas:
  - **t_S:** tiempo en segundos para realizar un seek
  - **t_T:** tiempo en segundos para transferir un bloque a memoria

Plan de ejecución

- A partir de las estadísticas y las estimaciones y a través de una heurística, se elige un **plan de ejecución** que consiste en el procedimiento completo incluyendo qué índices utilizar y cuáles algoritmos en cada caso.
- PostgreSQL (como la mayoría de los motores) nos permite inspeccionar el plan para una consulta:

https://explain.dalibo.com/

Estimación de costo

Estimación de costo - Selección

- La operación de selección (σ) puede implementarse con distintas estrategias según el contexto:
  - Búsqueda lineal (full scan)
  - Búsqueda con clustering index sobre atributo clave
  - Búsqueda con clustering index sobre atributo no clave
  - Búsqueda con secondary index sobre atributo clave
  - Búsqueda con secondary index sobre atributo no clave
  - Comparación (rango) con clustering index
  - Comparación (rango) con secondary index

Full Scan

- Implicará un seek inicial (para encontrar el primer bloque de la relación) y tantas transferencias como bloques tenga la relación:

$$t_S + b_r * t_T$$

Hace el seek + transferencia de b bloques a memoria

- En el caso excepcional en que la selección se realiza por igualdad con un atributo clave (es decir que *V(A, r)* es igual a la altura de r), en promedio será:

$$t_S + (b_r / 2) * t_T$$

Búsqueda con clustering index

- Si es sobre una clave, implicará recorrer la altura del árbol y después irá a buscar el registro al bloque de la relación:

$$(h_i + 1) * (t_T + t_S)$$

- Sobre no clave, será recorrer la altura del árbol, hacer un seek para encontrar el primer bloque que hace match y luego transferir los demás bloques (b) con registros para el valor:

$$h_i * (t_T + t_S) + t_S + b * t_T$$

Búsqueda con secondary index

- Si es sobre una clave, será equivalente al caso del clustering index:

$$(h_i + 1) * (t_T + t_S)$$

- Sobre no clave, implicará recorrer la altura del árbol y hacer un seek y un transfer para todos los registros (n) que hagan match:

$$(h_i + n) * (t_T + t_S)$$

Búsqueda por comparación (rango)

- Con un clustering index, será recorrer la altura del árbol, hacer un seek para encontrar el primer bloque que hace match y luego transferir los demás bloques (b) con registros de ese lado del rango:

$$h_i * (t_T + t_S) + t_S + b * t_T$$

  - Si la búsqueda es por < o <=, no es necesario recorrer el árbol:

$$t_S + b * t_T$$

- Con un índice secundario, habrá que recorrer la altura del árbol y hacer un seek y un transfer para todos los registros (n) que hacen match:

$$(h_i + n) * (t_T + t_S)$$

Selecciones complejas

Cuando la selección implica múltiples condiciones, hay distintas estrategias:

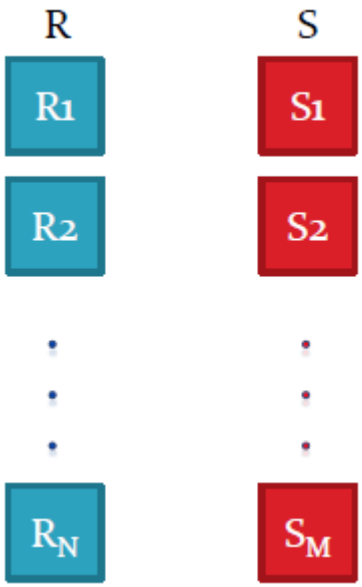
- Para conjunción de condiciones (and):
  - Utilizar el **índice para una de las condiciones**, aquella que tenga el menor costo. En este caso se utilizará alguno de los algoritmos anteriores y se evalúan las demás condiciones con los registros ya en memoria.
  - Utilizar un **índice compuesto** que aplique a todas las condiciones, aplicando una de las estrategias ya vistas.
  - Utilizar **índices para múltiples condiciones** e intersecar los identificadores.
- Para disyunción de condiciones (or):
  - Utilizar **índices para todas las condiciones** y hacer la unión de los identificadores.
  - Realizar un **full scan** y evaluar las condiciones en cada tupla en memoria.

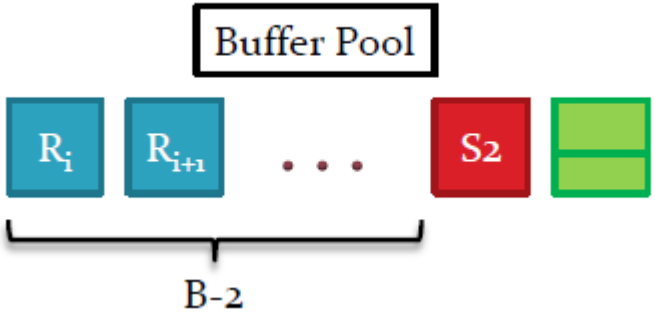
Junta

- La **junta** también cuenta con múltiples estrategias entre las que elegir:
  - Loop anidado
  - Loop anidado por bloques
  - Loop anidado indexado
  - Sort-merge
  - Hash

Junta - Loop anidado por bloques

Block Nested Loops Join (BNLJ)
B bloques de memoria



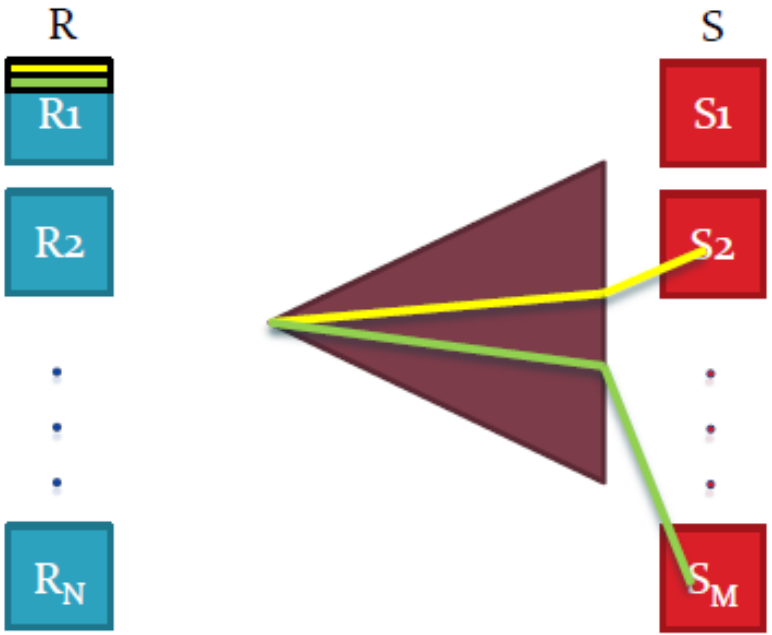


Junta - Loop anidado por bloques

- En los loops anidados, se recorren tuplas de la relación exterior (e) y se las compara con las tuplas de la relación interior (i)
- El loop anidado por bloques es una optimización sobre el loop anidado tradicional:
  - Se recorren los bloques de e.
  - Por cada bloque de e, se recorren los bloques de i.
  - Por cada bloque de i se comparan todas sus tuplas con las del bloque de e.
- Su costo será: $\mathbf{b_e * (1 + b_i) * (t_T + t_S)}$
- Por lo tanto habrá que elegir las relaciones para que $b_e < b_i$

Junta - Loop indexado

Index Nested Loops Join (INLJ)
Índice I sobre S



Junta - Loop indexado

- Al igual que en el caso por bloques, se itera una relación exterior (*e*) y una interior (*i*).
- Por cada tupla de *e* se busca la tupla de *i* correspondiente a través del **índice** del atributo de *i* involucrado en la junta.
- El peor caso implica que en memoria sólo se puede mantener un bloque de *e* (por el cual se iteran sus tuplas) y un bloque del índice.
- Sabemos que el costo (c) de la búsqueda de un valor por índice será:
  - Para atributo clave: $(h_i + 1) * (t_T + t_S)$
  - Para atributo no clave: $(h_i + n) * (t_T + t_S)$
- El costo entonces será: $b_e * (t_T + t_S) + n_e * c$

Junta - Merge-sort

Sort Merge Join (SMJ)
B bloques de memoria

R	
1	A
2	B
...	...
1000	ABC

S	
1	x
1	y
...	...
100	xyz

Junta - Merge-sort

- Consiste en **ordenar las tuplas** de cada relación a partir de los atributos de la condición de junta.
- Si las relaciones no entran en memoria, se hace un sort externo. Su costo (ordenar y volver a escribir), para una capacidad de memoria M, es:

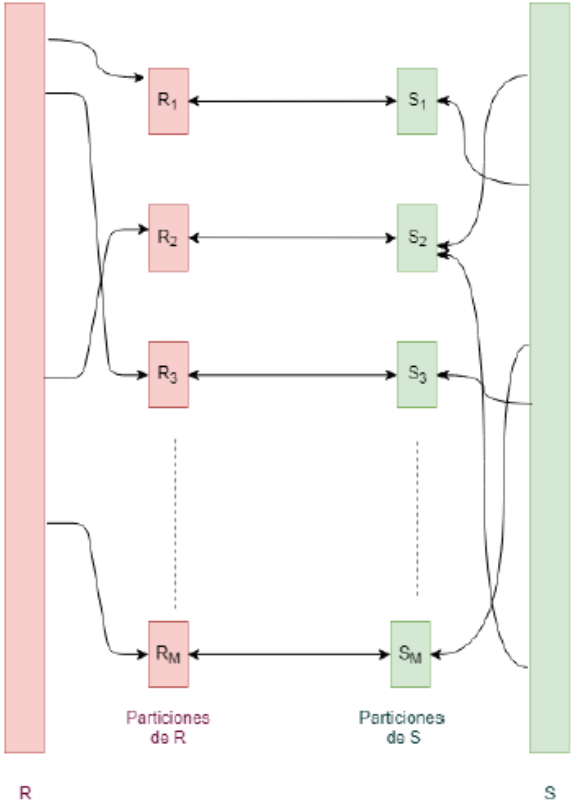
$$(2 * b_r * \lceil \log_{M-1}(b_r) \rceil) * (t_T + t_S)$$

- El costo total, incluyendo el merge posterior será:

$$(b_i + b_e + 2 * b_e * \lceil \log_{M-1}(b_e) \rceil + 2 * b_i * \lceil \log_{M-1}(b_i) \rceil) * (t_T + t_S)$$

Junta - Hash

Hash Join (HJ)



Junta - Hash

- En este algoritmo, se utiliza una función de **hash** sobre el conjunto de atributos de la condición de junta.
- A partir de esta función se van a dividir las tuplas de ambas relaciones en **particiones**.
- Asumiendo que la función está balanceada y que la distribución de valores de esos atributos es uniforme, las particiones van a ser parejas.
- Estas particiones se materializan para después hacer la junta. El costo del particionado, será el de leer todos los bloques de ambas relaciones y después escribirlos de nuevo:

$$2 * (b_e + b_i) * (t_T + t_S)$$

Junta - Hash

- Con las particiones ya generadas, será necesario comparar sólo aquellas en las que el valor del hash sea el mismo para las dos relaciones.
- Se hace la junta con cada par, verificando la condición por fuerza bruta.
- Mientras esto pueda hacerse (por cada par de grupos) en la memoria disponible, el costo de la combinación implicará leer todos los bloques de las particiones, lo que es equivalente a b_r:

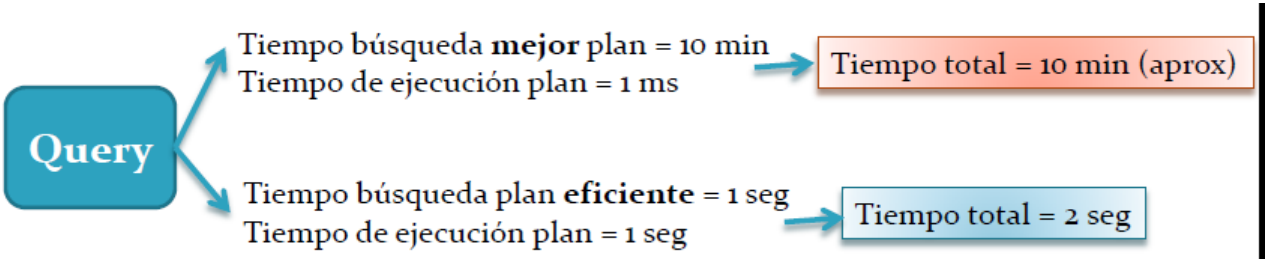
$$(b_e + b_i) * (t_T + t_S)$$

- Por lo tanto, el costo total será:

$$3 * (b_e + b_i) * (t_T + t_S)$$

Optimización

Plan de ejecución → algoritmos y estructuras



¿Plan eficiente o mejor plan?

Analizar todo el espacio de búsqueda → costoso

Evitar analizarlo completo, pero obtener planes eficientes

Selección del plan de ejecución ¿óptimo?

- El optimizador busca el plan de ejecución “óptimo” a partir de comparar distintas **expresiones de álgebra relacional equivalentes** entre sí (y al SQL).
- Para lograr esto se va apoyar en:
  - **Reglas de equivalencia:** para generar las expresiones posibles
  - **Heurísticas de optimización:** para reducir las posibilidades a evaluar
  - **Análisis de costo**: el cálculo de costo para cada tipo de operador
  - **Estimación de cardinalidades:** para las relaciones intermedias

Reglas de equivalencia

- Las reglas de equivalencia permite transformar expresiones en otras equivalentes:
- Selección:
  - Las condiciones conjuntivas (and) pueden explotarse en condiciones individuales encadenadas: $\sigma_{\theta 1 \wedge \theta 2}(E) \equiv \sigma_{\theta 1}(\sigma_{\theta 2}(E))$
  - Las disyunciones pueden explotarse en uniones: $\sigma_{\theta 1 \vee \theta 2}(E) \equiv \sigma_{\theta 1}(E) \cup \sigma_{\theta 2}(E)$
  - Las condiciones encadenadas son conmutativas: $\sigma_{\theta 1}(\sigma_{\theta 2}(E)) \equiv \sigma_{\theta 2}(\sigma_{\theta 1}(E))$

Reglas de equivalencia

- Operaciones de conjunto:
  - La unión, la intersección y la junta son conmutativas: $E_1 \cup E_2 \equiv E_2 \cup E_1$
  - Y asociativas: $(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$
- Junta:
  - La junta theta es asociativa mediante:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 \equiv E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

Heurísticas

- Una serie de reglas heurísticas permiten reducir el espacio de expresiones a comparar:
  - Realizar las selecciones lo antes posible.
  - Realizar las proyecciones (dado que reducen cardinalidad) temprano.
  - Realizar las juntas más restrictivas primero
  - Etc…
- Los optimizadores combinarán decisiones basadas en costo con decisiones basadas en heurísticas para diferentes partes de la misma expresión

Procesamiento de consultas

- Dada la consulta:

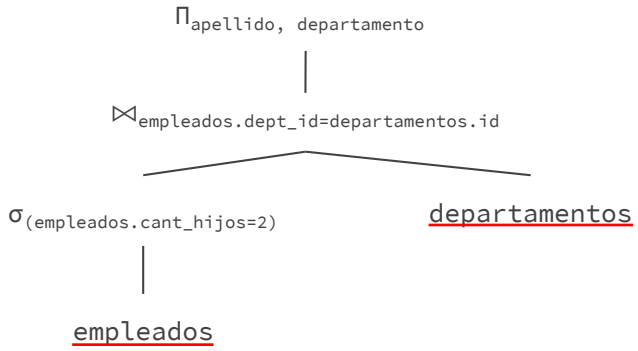
```
SELECT apellido, departamento FROM empleados
INNER_JOIN departamento
ON(empleados.dept_id = departamento.id)
WHERE empleados.cant_hijos = 2
```

- ¿Cuáles son las posibles expresiones de álgebra relacional?
- ¿De qué formas podemos resolver las distintas partes de esas expresiones en una base de datos?

Evaluación de expresiones de álgebra

- Las expresiones de álgebra se evalúan construyendo un árbol de operadores:

$$\pi_{apellido}(\sigma_{empleados.cant_hijos=2}(\underline{empleados}) \bowtie_{empleados.dept_id=departamentos.id} \underline{departamentos})$$



Evaluación de expresiones de álgebra

- El árbol se evalúa desde sus hojas y hacia su raíz.
- Cada nodo implica una expresión cuyo resultado es una relación
- Si estos resultados intermedios no entran en memoria, se deben materializar (escribir a disco). Esto implica costos:
  - Habrá que escribir tantos bloques como n_r/f_r (donde r es la relación intermedia)
  - Los operadores siguientes en el árbol, implicarán leer esos bloques también

¿Cómo se pasan los resultados entre nodos del árbol de operadores?

Materialización

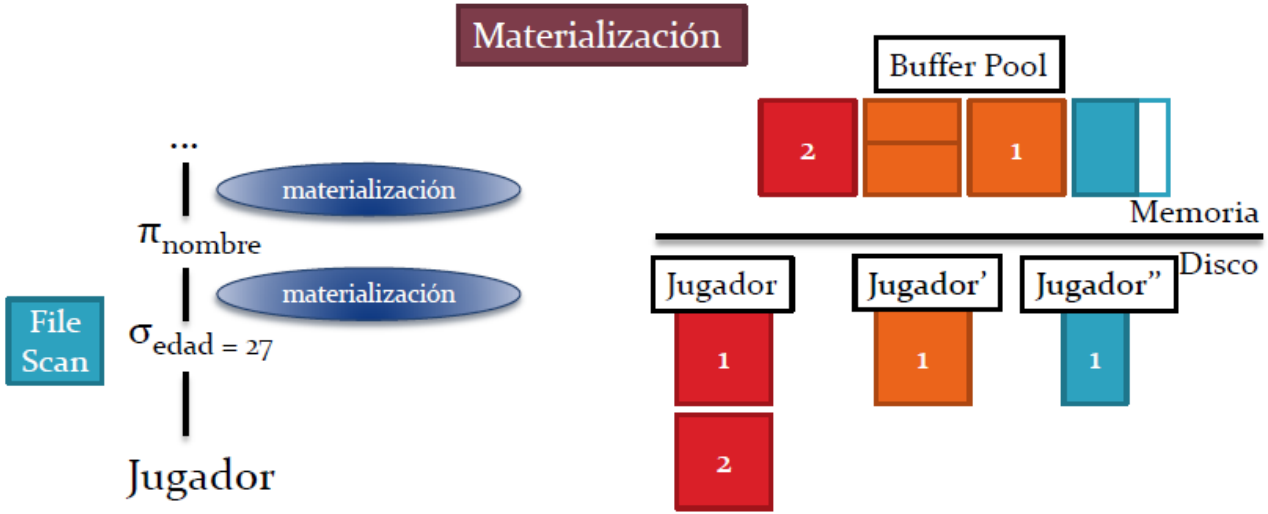
Los resultados intermedios se guardan **directamente en disco**.

El siguiente paso de la consulta deberá levantarlos de disco **nuevamente**.

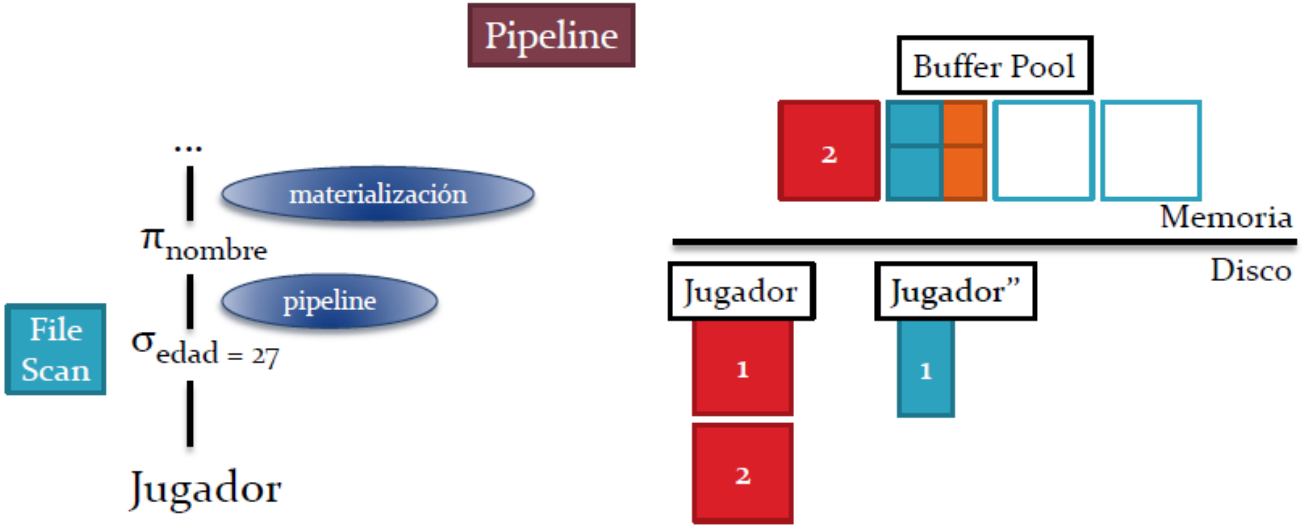
Pipeline

Las tuplas se van pasando al nodo superior del árbol **mientras** se continúa ejecutando la operación

Materialización



Pipeline


Pipeline
...
materialización
π_{nombre}
pipeline
File Scan
$\sigma_{edad = 27}$
Jugador
Buffer Pool
2
Memoria
Disco
Jugador
Jugador"
1
1
2

Pipelining

- La evaluación por **pipelining** consiste en encadenar operadores de tal manera que los resultados de uno pueden ser procesados por el siguiente incluso con **resultados parciales**.
- Esto reduce la cantidad de **archivos temporales** a generar (y leer) cuando los resultados no entran en la memoria disponible.
- Además, en ciertos casos, puede **reducir la latencia**, permitiendo que se presenten los primeros resultados antes de que se termine de procesar la totalidad.

Estimación de cardinalidades

- Las **estimaciones de costo** están basadas en **estadísticas de las relaciones**, pero muchas veces la relación involucrada en un operador será el resultado de otra expresión compleja.
- Para poder calcular el costo en estos casos, hay que poder estimar la cardinalidad de las relaciones intermedias.

Estimación de cardinalidades

▨ Selección:

- **$\sigma_{A=a}(r)$**: podemos estimar $n_r/V(A, r)$
- **$\sigma_{A\leq v}(r)$**:
  - si $v < \min(A,r)$: 0;
  - si $v > \max(A,r)$: n_r;
  - si no: $n_r(v - \min(A,r)) / (\max(A,r) - \min(A,r))$
- **$\sigma_{\theta 1 \wedge \theta 2 \wedge \cdots \wedge \theta n}(r)$**: la cardinalidad de cada selección se calcula como vimos más arriba. Para condiciones independientes, si s_1 es el tamaño de la sección θ_1, entonces la cardinalidad total será: $n_r * s_1 * s_2 * \ldots * s_n / n_r^n$
- **$\sigma_{\theta 1 \vee \theta 2 \vee \cdots \vee \theta n}(r)$**: la cardinalidad será: $1 - (1 - s_1/n_r)*(1 - s_1/n_r)* \ldots *(1 - s_n/n_r)$

Estimación de cardinalidades

- **Junta:**
  - **$r \times s$:** $n_r * n_s$
  - **$r \bowtie_\theta s$ si θ implica clave en s:** n_s (como máximo)
  - **$r \bowtie_\theta s$ si θ implica clave en r:** n_r (como máximo)
  - **$r \bowtie_\theta s$ si θ no implica clave:**
    - por cada tupla de una de las relaciones (r) se producen n_s / V(A,s)
    - por lo tanto, en total, visto desde r se producen $n_r * n_s$ / $V(A,s)$
    - se asume el min entre $n_r * n_s$ / $V(A,s)$ y $n_s * n_r$ / $V(A,r)$

Estimación de cardinalidades

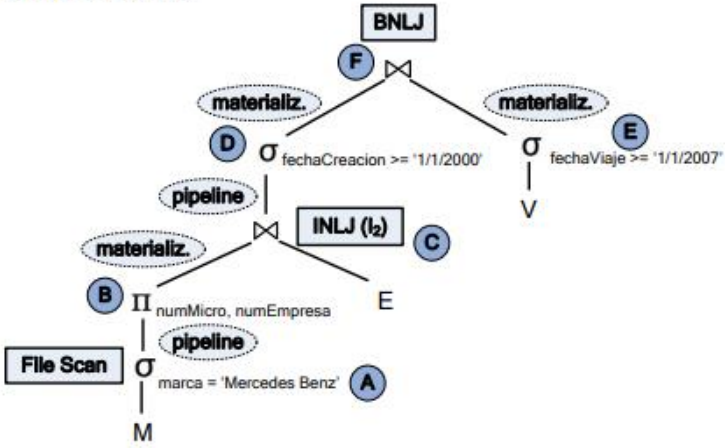
- Proyección (distinct)
  - $\pi_A(r)$: la cardinalidad será $V(A,r)$ (dado que no hay duplicados)
- Agregación
  - ${}_G\gamma_A(r)$: la cardinalidad será $V(G,r)$ (una fila por cada valor de G)
- Operaciones de conjunto
  - $r \cup s$: será $n_r + n_s$
  - $r \cap s$: se estimará como $min(n_r, n_s)$
  - $r \ltimes s$: será como máximo la estimación de $r \bowtie s$, más n_r
  - $r \rtimes s$: será como máximo la estimación de $r \bowtie s$, más n_r, más n_s

Ejemplo

Query Editor Query History

```sql
SELECT
        numMicro
        , numEmpresa
        , nombreEmpresa
        , fechaCreacion
        , direccion
        , destino
        , fechaViaje
FROM
    Viaje V
    , Empresa E
    , Micro M
WHERE
    M.numMicro = V.numMicro
    AND M.numEmpresa = E.numEmpresa
    AND E.fechaCreacion >= '1/1/2000'
    AND V.fechaViaje >= '1/1/2007'
    AND V.fechaViaje < '1/2/2007'
    AND M.marca = 'Mercedes Benz'
```

Calcular los costos:



Cierre

Resumen

- El **procesamiento de consultas** en un motor de base de datos relacional implica distintos pasos: traducción a álgebra, búsqueda de expresiones equivalentes, cálculo de costos, selección del plan óptimo.
- La búsqueda de expresiones alternativas se realiza a partir de **reglas de equivalencia y heurísticas**.
- La evaluación de costos se realiza en base a la cantidad de **seeks y transfers** de I/O que se van a realizar.
- Contamos con reglas para **estimar los costos** de los distintos tipos de expresiones y para **calcular las cardinalidades** de las relaciones intermedias.
- Mediante la técnica de **pipelining** se puede ir procesando el resultado de expresiones intermedias a medida que se va obteniendo.